Tool Implementation

Modulo 3 · Lezione 8



Una domanda sui dati, senza un tool che li legge

il file della lezione scorsa · tool: `calcolatrice`, `data_oggi`

`«Quanti ordini ha fatto Giulia Moretti?»`

> Puoi fornirmi i dati o il contesto relativo agli ordini di Giulia Moretti?

5 volte su 5: chiede i dati



`dataset/lumen.db`

il database degli ordini Lumen

13

gli ordini di Giulia Moretti

Il modello non inventa: sa di non avere i dati. Nessuno dei suoi tool li legge.

> Stasera: un tool che **legge il database**.

Agenda

	Blocco	Dove	Tempo
	Apertura: la domanda sui dati	slide	5′
	Il database degli ordini	slide	10′
	Demo del docente: tre query a mano	`scaffolding/sql.py`	5′
	Cosa cambia quando un tool tocca dati veri	slide	20′
	Codice spiegato: `query_db` (già nel file)	`working/tools.py`	15′
	Esercizio: `prezzo_prodotto` e i casi rotti (TODO)	`working/tools.py`	60′
	Recap e prossima lezione	slide	5′

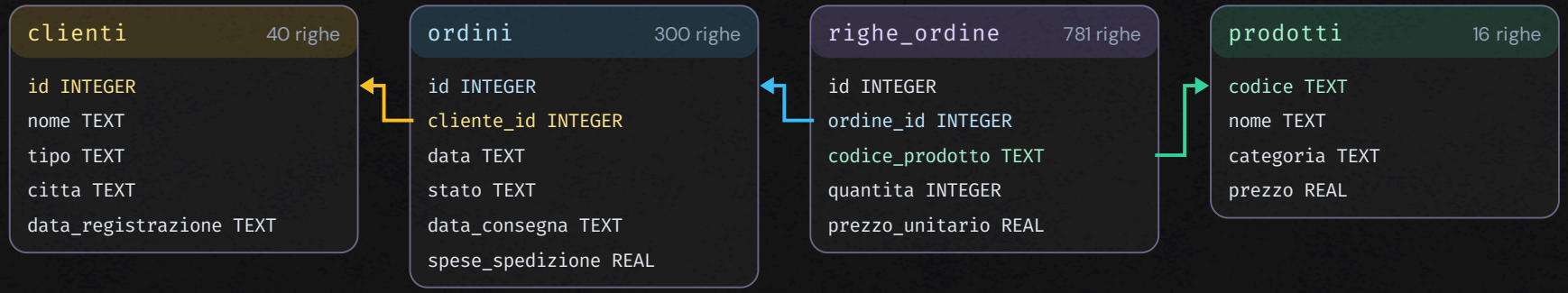
Il file è `working/tools.py`, lo stesso della lezione scorsa, nella cartella `scripts/` di questa lezione. Il database è `dataset/lumen.db`.

Setup come la lezione scorsa, con un venv nuovo in questa cartella: `pip install -r requirements.txt`, la chiave in `scripts/.env` (copiate il `.env` della lezione scorsa). Nessuna libreria nuova: `sqlite3` è già in Python.



Il database degli ordini: quattro tabelle

`dataset/lumen.db` è un file **SQLite**: niente server, Python lo legge con la libreria standard. Le frecce sono le colonne che collegano le tabelle.



un ordine è di un cliente · una riga d'ordine è di un ordine e di un prodotto

privati e aziende, con la città

da gennaio ad agosto 2026

un prodotto per riga, con la quantità

i codici del listino che il RAG conosce

Una riga per tabella

prodotti

codice	nome	categoria	prezzo
LMX-4400	Braccio porta monitor…	accessori	89.0

clienti

id	nome	tipo	citta	data_registrazione
1	Giulia Moretti	privato	Padova	2025-06-25

ordini

id	cliente_id	data	stato	data_consegna	spese_spedizione
1286	1	2026-08-16	consegnato	2026-08-22	0.0

righe_ordine dell'ordine 1286

ordine_id	codice_prodotto	quantita	prezzo_unitario
1286	LC-100	1	349.0
1286	LD-115	1	289.0
1286	LD-200	1	699.0
1286	LMX-4410	1	149.0

cliente_id 1 dell'ordine è Giulia Moretti.

ordine_id 1286 delle righe è quell'ordine: una seduta, due piani e un braccio doppio.



Quattro dettagli delle colonne

1 · le date

Sono **testo**, nel formato AAAA-MM-GG: `2026-08-16`.

2 · stato, quattro valori



`reso 31` `annullato 23` `spedito 5`

3 · data_consegna

È **vuota** (`NULL`) se l'ordine non è arrivato: spedito o annullato.

4 · due prezzi

«quanto costa?»
`prodotti.prezzo`
il listino di oggi

«quanto ha speso?»
`righe_ordine.prezzo_unitario`
il prezzo pagato

Il database si apre **in sola lettura**: stasera nessuno scrive.

Tre query da leggere

```
SELECT * FROM ordini LIMIT 5
```

i primi cinque ordini

```
SELECT r.codice_prodotto, p.nome, r.quantita
FROM righe_ordine r JOIN prodotti p ON p.codice = r.codice_prodotto
WHERE r.ordine_id = 1286
```

cosa c'è nell'ordine 1286, con i nomi dei prodotti

```
SELECT COUNT(*)
FROM ordini o JOIN clienti c ON c.id = o.cliente_id
WHERE c.nome = 'Giulia Moretti'
```

quanti ordini ha fatto Giulia Moretti

`SELECT` le colonne	`FROM` la tabella	`JOIN … ON` unisce due tabelle sulla colonna che le collega	`WHERE` filtra le righe	`COUNT` conta	`LIMIT` taglia il risultato

Da stasera le query le scrive il modello: vanno saputе **leggere**, per capire se sono giuste.



Demo del docente: le tre query a mano

DEMO DEL DOCENTE — il docente lancia, chi vuole lo lancia anche lui

`scaffolding/sql.py · niente modello · 5 minuti`

si rientra dopo la terza query

```
# dalla cartella scripts/
python scaffolding/sql.py
  → le tabelle, le colonne, quante righe

python scaffolding/sql.py "SELECT * FROM ordini LIMIT 5"
python scaffolding/sql.py "SELECT r.codice_prodotto, … WHERE r.ordine_id = 1286"
python scaffolding/sql.py "SELECT COUNT(*) FROM ordini o JOIN clienti c … "

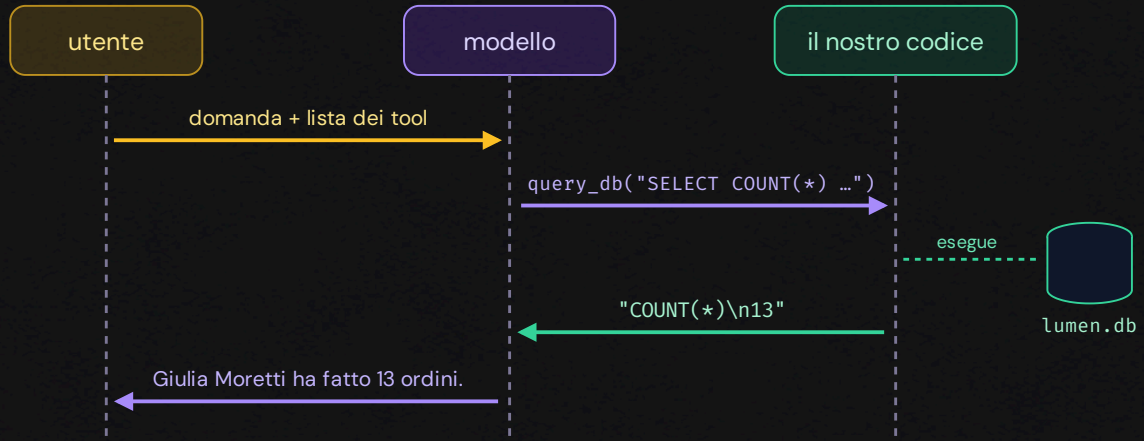
COUNT(*)
────────
13
```

13: la risposta alla domanda dell'apertura, **scritta da noi**.

tra poco la query la scrive il modello



Text-to-SQL: il modello scrive, noi eseguiamo



lo stesso giro della lezione scorsa: cambia cosa fa il tool

Il tool nuovo è `query_db(sql)`, con un solo argomento: la **query**. Da una domanda in italiano a una query SQL: si chiama **text-to-SQL**.

la query la **scrive il modello**

la **esegue il nostro codice**, che decide cosa è permesso

Cosa vede il modello: lo schema, non i dati

working/tools.py · la docstring di QueryDb è la descrizione del tool

```
class QueryDb(BaseModel):
    """Esegue una query SQL (solo SELECT, dialetto SQLite) sul database degli
    ordini Lumen e ritorna le righe trovate. Tabelle e colonne:
    prodotti(codice TEXT, nome TEXT, categoria TEXT, prezzo REAL)
    clienti(id INTEGER, nome TEXT, tipo TEXT, citta TEXT, data_registrazione TEXT)
    ordini(id INTEGER, cliente_id INTEGER, data TEXT, stato TEXT, data_consegna TEXT, spese_spedizione REAL)
    righe_ordine(id INTEGER, ordine_id INTEGER, codice_prodotto TEXT, quantita INTEGER, prezzo_unitario REAL)"""
    sql: str
```

VEDE

tabelle, colonne, tipi. Di un tool conosce nome, descrizione e parametri: lo schema sta lì.

NON VEDE

i valori. Sa che c'è `stato`, non sa cosa c'è dentro.

+270 token a ogni chiamata

15 su 15 risposte giuste quando il modello non deve indovinare un valore

quando deve indovinare un valore, no



Gli errori tornano al modello

Una query sbagliata fa sollevare un'eccezione: se sale, il giro si ferma e il programma muore. `query_db` la prende e la **restituisce come testo**. Un giro registrato, con il solo `query_db` tra i tool:

«Quanti giorni ci vogliono in media tra ordine e consegna?»

```
1 · query_db SELECT ordine_id, data, data_consegna FROM ordini WHERE data_consegna IS NOT NULL
    → "Errore: no such column: ordine_id"
```

```
2 · query_db SELECT id, data, data_consegna FROM ordini WHERE data_consegna IS NOT NULL
    → "id | data | data_consegna\n1003 | 2026-01-05 | 2026-01-11 …" (50 righe)
```

```
3 · query_db SELECT AVG(julianday(data_consegna) - julianday(data)) FROM ordini WHERE …
    → "giorni_medi\n4.974264705882353"
```

```
4 · message · In media ci vogliono circa 5 giorni tra l'ordine e la consegna.
```

```
IN query_db
except Exception as e:
    return f"Errore: {e}"
```

Quattro chiamate invece di due: lo stesso ciclo della lezione scorsa, ripetuto. Non succede spesso: quando la domanda contiene il valore, la query è giusta al primo colpo.

`julianday` trasforma una data in un numero di giorni: la differenza è la durata.



Sicurezza: i controlli stanno nel codice

sola lettura

```
sqlite3.connect(DB.as_uri() + "?mode=ro", uri=True)
```

il file non si può modificare. Senza `mode=ro`, se il file manca, `sqlite3` ne crea uno vuoto.

solo SELECT

```
if not sql.strip().lower().startswith("select"):
```

una query che non comincia così non parte, e il modello riceve un messaggio chiaro. La sola lettura vera la garantisce `mode=ro`.

al massimo LIMITE righe

```
righe = cursore.fetchmany(LIMITE + 1)
```

al modello arrivano 50 righe al massimo, con una nota se erano di più.

al massimo MAX_CHIAMATE al modello

```
if n_chiamate == MAX_CHIAMATE:
```

con gli errori che tornano al modello, il giro potrebbe non finire: si ferma a 6.

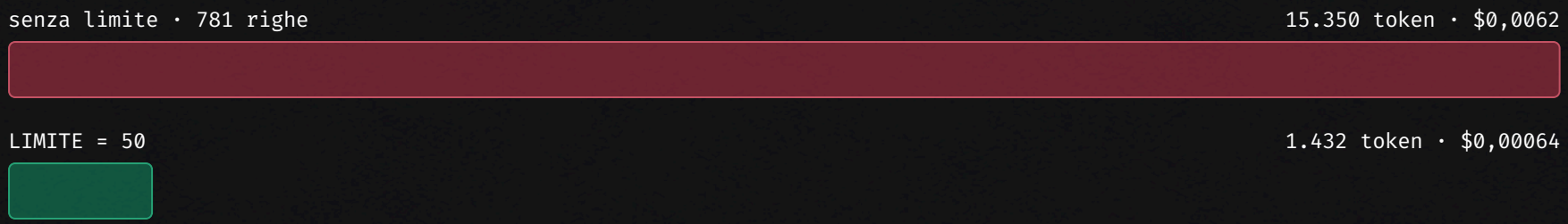
il modello: **0 tentativi di scrittura** su 15 richieste di cancellare o modificare

la descrizione il modello può ignorarla, i controlli li fa **il codice**



Il costo di un output senza limite

Quello che il tool restituisce entra nell'input della chiamata dopo, e si paga. `SELECT * FROM righe_ordine` sono 781 righe:



Dieci volte meno. E il modello il `LIMIT` **non lo mette da solo**: su «Mostrami tutte le righe d'ordine» ha scritto `SELECT *` ogni volta.

Il taglio lo fa `query_db`, e lo scrive in fondo al risultato:

```
(mostrate solo le prime 50 righe)
```

Query valide, risposte sbagliate

Queste **non danno errore**: SQL valida, eseguita bene, su valori che il database non contiene.

domanda	il modello scrive	risponde	nel database
Quanti ordini sono ancora in viaggio?	`stato = 'in viaggio'`	nessuno (10 su 10)	5 · `'spedito'`
Quanti ordini sono stati cancellati?	`stato = 'cancellato'`	nessuno	23 · `'annullato'`
Quanti ordini ha fatto il cliente Rossi?	`nome = 'Rossi'`	nessuno	15 · Marco e Lorenzo Rossi: domanda ambigua
Quali clienti hanno ordinato la LMX-4400 a giugno?	`'2023-06'`	nessun ordine	gli ordini sono del 2026 (non usa data_oggi, che ha)

La risposta sbagliata esce **con sicurezza**. Il motivo: il modello vede le colonne, non i valori.

`ERRORI`

il codice li vede, e li rimanda al modello

`QUERY VALIDE MA SBAGLIATE`

il codice non le vede: la cura è Modulo 3 · Lezione 13



La struttura di working/tools.py

`DB · LIMITE · MAX_CHIAMATE` nuovo	il file del database, le righe massime, le chiamate massime
1 `Calcolatrice · DataOggi`	come le avete lasciate la lezione scorsa
2 `QueryDb + query_db(sql)` nuovo	lo schema nella docstring e la query eseguita: il **codice spiegato**
`TODO PrezzoProdotto + prezzo_prodotto()`	il prezzo di listino dato il codice: lo scrivete voi
3 `definisci_tool() + TOOLS`	come prima: la lista che il modello vede
4 `esegui(nome, argomenti)`	un ramo in più: `query_db`
5 `chiedi(domanda)`	il giro, con il tetto `MAX_CHIAMATE`
`helper di stampa · main`	il risultato di un tool ora su più righe · tre DOMANDE, o SOLO



Codice spiegato: query_db

CODICE SPIEGATO — il docente spiega, voi seguite sul vostro file

`working/tools.py · tutto già scritto tranne il TODO · 15 minuti`

si rientra dopo il lancio, con l'output a schermo

`2 · query_db(sql), le righe su cui ci si ferma`

```
if not sql.strip().lower().startswith("select"): …
con = sqlite3.connect(DB.as_uri() + "?mode=ro", uri=True)
righe = cursore.fetchmany(LIMITE + 1)
except Exception as e: return f"Errore: {e}"
```

poi la docstring di QueryDb, il ramo in `esegui()`, il tetto in `chiedi()`

`il lancio: tre domande`

1 · Quanti ordini ha fatto Giulia Moretti?
2 · Quanti ordini sono ancora in viaggio?
3 · Quanto costa la LMX-4400?

Poi lo lancia il docente, dalla cartella `scripts/`: `python working/tools.py`. Chi ha già `.env` pronto lo lancia anche lui.



Cosa si è visto: tre domande al database

```
D: Quanti ordini ha fatto Giulia Moretti?
   chiamata 1  [432 token in, 45 out]  output:
     function_call query_db({"sql":"SELECT COUNT(*) … WHERE clienti.nome = 'Giulia Moretti';"})
      eseguo query_db: numero_ordini
         13
   chiamata 2  [490 token in, 13 out]  output: message · 2 chiamate · $0.00046
→ Giulia Moretti ha fatto 13 ordini.

D: Quanti ordini sono ancora in viaggio?
     function_call query_db({"sql":"SELECT COUNT(*) … WHERE stato = 'in viaggio'"})
      eseguo query_db: ordini_in_viaggio
         0
→ Non ci sono ordini attualmente in viaggio.  sono 5

D: Quanto costa la LMX-4400?
     function_call query_db({"sql":"SELECT prezzo FROM prodotti WHERE nome = 'LMX-4400'"})
      eseguo query_db: prezzo ← nessuna riga
→ Non risulta un prezzo associato al prodotto LMX-4400 nel database.  89 euro
```

1 · giusta: la query della demo a mano, scritta dal modello.

2 · sbagliata, nessun errore: il valore è `'spedito'`. Come si cura: Modulo 3 · Lezione 13.

3 · sbagliata 7 volte su 10: il codice è nella colonna `codice`.
Nell'esercizio: **un tool nuovo**, con la query scritta da voi.



Esercizio: prezzo_prodotto e i tre casi rotti

SCRIVETE VOI — da soli, con affiancamento

`working/tools.py` · 60 minuti · traccia ed extra in CONSEGNA.md

si rientra per il recap

20' il TODO · 10' la domanda sul prezzo · 20' i tre casi rotti · il resto l'extra

`il TODO: prezzo_prodotto(codice)`

1 · la docstring di `PrezzoProdotto` e il corpo di `prezzo_prodotto()`, già abbozzati

2 · la voce in `TOOLS`

3 · il ramo in `esegui()`

Il modello passa il codice; la query la scrivete voi, con il segnaposto. Mai il codice incollato nella stringa: il testo del modello cambierebbe la query.

```
con.execute("… WHERE codice = ?", (codice,))
```

`poi, un lancio per riga, con SOLO in cima al file`

SOLO = …	risposta attesa
`"Quanto costa la LMX-4400?"`	quale tool usa? e perché?
`1 · "Quanto costa la LMX-9999?"`	non è nel listino
`2 · "Quanto costa il codice lmx-4400, scritto esattamente così?"`	89 euro, con prezzo_prodotto che riceve lmx-4400 (nel codice: maiuscolo, senza spazi ai lati)
`3 · lumen.db rinominato in lumen-vecchio.db, poi "Quanto costa la LMX-4400?"`	il listino non si raggiunge; poi cancellate un lumen.db vuoto, se c'è, e solo dopo rimettete il nome

`FATTO QUANDO`

1 · Sui tre casi rotti il programma non muore e il modello dà la risposta attesa.

2 · Su «Quanto costa la LMX-4400?» sapete dire quale tool ha usato il modello, e perché.



Cosa si è visto: la query fissa e i casi rotti

«Quanto costa la LMX-4400?»

solo `query_db`

3 su 10

giuste: di solito `WHERE nome = …`

con `prezzo_prodotto`

10 su 10

sceglie `prezzo_prodotto`, 89 euro · a voi basta un lancio

Lo sceglie perché nome e descrizione dicono «prezzo di listino, dato il codice». Risponde giusto perché la query l'avete scritta voi.

caso rotto	versione minima, senza controlli	con i controlli
`LMX-9999`	`TypeError: 'NoneType' object is not subscriptable`	«non è presente nel listino»
`lmx-4400`	lo stesso `TypeError`: il codice minuscolo non c'è	89 euro
`database rinominato`	`no such table: prodotti`, e un `lumen.db` vuoto creato da `connect()` (con `mode=ro`: `unable to open database file`)	«non posso accedere al listino»

Senza controlli il programma muore in tutti e tre i casi; con il solo controllo sul `None`, il codice minuscolo dà «non è nel listino». Con i controlli il modello riceve un messaggio e risponde: due chiamate, nessuna eccezione. Il `lumen.db` vuoto rimasto dalla prova va cancellato prima di rimettere il nome giusto.

Nei sistemi veri i codici arrivano sporchi: minuscoli, con spazi, senza trattino. `strip()` e `upper()` coprono i primi due, ma su `lmx4400` non bastano: lì si usa una ricerca per somiglianza. Non la scriviamo stasera.



Recap

1. Lo **schema** va nella descrizione del tool; i dati il modello non li vede.
2. Gli **errori** di `query_db` e `prezzo_prodotto` tornano al modello come testo, invece di far morire il programma.
3. Sola lettura, solo SELECT, limite alle righe e alle chiamate: **nel codice**.
4. Quando la domanda ha sempre la stessa forma, un tool con la **query fissa**: `WHERE codice = ?`

? Resta aperto: le query valide che danno la risposta sbagliata → Modulo 3 · Lezione 13.

Stasera il tool legge un database. Allo stesso modo si scrivono tool che chiamano un servizio web, come una ricerca o il meteo, con le stesse regole.

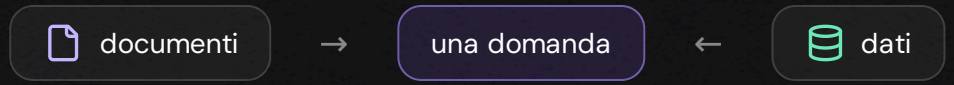


La prossima lezione

stasera
i dati: `query_db` e `prezzo_prodotto`

→

Modulo 3 · Lezione 9 — Lab: RAG + Tools
il RAG diventa un tool accanto a `query_db` e `prezzo_prodotto`, e il sistema sceglie se la risposta sta nei documenti o nei dati



Grazie

prossima lezione: Modulo 3 · Lezione 9 — Lab: RAG + Tools

